

AVL Tree Rotations & Operations — Data Structures

Definition: An **AVL Tree** (Adelson-Velsky and Landis) is a **self-balancing Binary Search Tree** in which the **Balance Factor (BF)** of every node is either **-1, 0, or +1**.

1. Detailed Technical Explanation

Balance Factor Formula:

Balance Factor (BF) = Height(Left Subtree) - Height(Right Subtree)
Valid AVL Condition: $BF \in \{-1, 0, +1\}$

If after an insertion or deletion, the balance factor of any node becomes ≤ -2 or $\geq +2$, the tree is **unbalanced** and must be rebalanced using **Rotations**.

2. The Four AVL Tree Rotations

1. LL Rotation (Single Right Rotation)

- Cause:** Insertion into the **Left subtree of the Left child** of node **z** (BF of **z** = +2, BF of left child = +1).
- Fix:** Perform a single Right Rotation at **z**.

```

      [ z ] (+2)           [ y ] (0)
      /                   /  \
    [ y ] (+1)         [ x ]  [ z ]
      /               Rotation at z
    [ x ]
  
```

2. RR Rotation (Single Left Rotation)

- Cause:** Insertion into the **Right subtree of the Right child** of node **z** (BF of **z** = -2, BF of right child = -1).
- Fix:** Perform a single Left Rotation at **z**.

```

    [ z ] (-2)           [ y ] (0)
      \                   /  \
    [ y ] (-1)         [ z ]  [ x ]
      \               Rotation at z
    [ x ]
  
```

3. LR Rotation (Double Rotation: Left then Right)

- Cause:** Insertion into the **Right subtree of the Left child** of node **z** (BF of **z** = +2, BF of left child = -1).
- Fix:** Perform **Left Rotation** at **y**, followed by **Right Rotation** at **z**.

$$\begin{array}{ccc}
 \begin{array}{c} [z] (+2) \\ / \\ [y] (-1) \\ \backslash \\ [x] \end{array} & \begin{array}{c} == \text{Left} \Rightarrow \\ \text{on } y \end{array} & \begin{array}{c} [z] (+2) \\ / \\ [x] \end{array} \\
 & & \begin{array}{c} == \text{Right} \Rightarrow \\ \text{on } z \end{array}
 \end{array}
 \begin{array}{ccc}
 \begin{array}{c} [x] (0) \\ / \quad \backslash \\ [y] \quad [z] \end{array}
 \end{array}$$

4. RL Rotation (Double Rotation: Right then Left)

- **Cause:** Insertion into the **Left subtree of the Right child** of node **z** (BF of **z** = -2, BF of right child = +1).
- **Fix:** Perform **Right Rotation** at **y**, followed by **Left Rotation** at **z**.

3. Time Complexity of AVL Tree Operations

Operation	Average Case	Worst Case
Search	$O(\log N)$	$O(\log N)$
Insertion	$O(\log N)$ (at most 2 rotations)	$O(\log N)$
Deletion	$O(\log N)$ (up to $O(\log N)$ rotations)	$O(\log N)$

4. Must-Write Points for Exams

- The height of an AVL tree with **N** nodes is strictly bounded by $1.44 * \log_2(N)$.
- AVL tree provides faster lookup $O(\log N)$ than Red-Black tree because it is more strictly balanced.
- Insertion requires at most 1 single or double rotation; deletion may require up to $O(\log N)$ rotations propagating up the tree.

5. Quick Recall Flow

Balance Factor = $H(\text{Left}) - H(\text{Right}) \in \{-1, 0, 1\}$ -> Imbalance: LL (Right Rotate), RR (Left Rotate), LR (Left-Right), RL (Right-Left)